

Rate Monotonic Scheduling (RMS)

RTOS Explanation and Step-by-Step Example

Prashant Jadav
DEPSTAR, CHARUSAT

1 Introduction

Rate Monotonic Scheduling (RMS) is a fixed-priority, preemptive scheduling algorithm for periodic real-time tasks.

The fundamental rule is:

$$\boxed{\text{Shorter period} \Rightarrow \text{Higher priority}}$$

A periodic task may be represented as:

$$T_i = (C_i, P_i, D_i)$$

where C_i is execution time, P_i is period, and D_i is relative deadline. In the standard RMS model, it is commonly assumed that:

$$D_i = P_i.$$

2 RMS Utilization-Bound Test

Processor utilization is:

$$\boxed{U = \sum_{i=1}^n \frac{C_i}{P_i}}$$

The RMS sufficient utilization bound for n tasks is:

$$\boxed{U_{RMS} = n(2^{1/n} - 1)}$$

If

$$U \leq U_{RMS},$$

the task set is guaranteed schedulable under the standard assumptions.

For large n , the bound approaches:

$$\ln 2 \approx 69.3\%.$$

3 Example

Given:

Process	Execution Time C	Period T
$P1$	3	20
$P2$	2	5
$P3$	2	10

Assume all processes are released at $t = 0$, scheduling is preemptive, deadlines equal periods, and context-switch overhead is negligible.

4 Step 1: Determine RMS Priorities

The periods are:

$$T_{P1} = 20, \quad T_{P2} = 5, \quad T_{P3} = 10.$$

Since:

$$5 < 10 < 20,$$

the RMS priority order is:

$$\boxed{P2 > P3 > P1}.$$

Thus $P2$ has the highest priority and $P1$ the lowest.

5 Step 2: Calculate Processor Utilization

$$U = \frac{3}{20} + \frac{2}{5} + \frac{2}{10}$$

$$U = 0.15 + 0.40 + 0.20$$

$$\boxed{U = 0.75 = 75\%}.$$

6 Step 3: Calculate RMS Utilization Bound

There are $n = 3$ tasks:

$$U_{RMS} = 3(2^{1/3} - 1).$$

Since:

$$2^{1/3} \approx 1.2599,$$

$$U_{RMS} \approx 3(0.2599) = 0.7798.$$

Therefore:

$$\boxed{U_{RMS} \approx 77.98\%}.$$

Comparison:

$$75\% < 77.98\%.$$

Hence:

$$\boxed{\text{The task set passes the RMS utilization-bound test.}}$$

7 Step 4: Hyperperiod

The hyperperiod is:

$$H = \text{LCM}(20, 5, 10) = 20 \text{ ms.}$$

Therefore, the release pattern repeats every 20 ms.

8 Step 5: Job Releases and Deadlines

Since $D_i = P_i$:

Job	Release	C	Deadline
$P2_1$	0	2	5
$P3_1$	0	2	10
$P1_1$	0	3	20
$P2_2$	5	2	10
$P2_3$	10	2	15
$P3_2$	10	2	20
$P2_4$	15	2	20

9 Step 6: Construct the RMS Schedule

At $t = 0$, all three processes are ready. Since:

$$P2 > P3 > P1,$$

execute $P2$:

$$0 \rightarrow 2 : P2_1.$$

Then execute $P3$:

$$2 \rightarrow 4 : P3_1.$$

Then $P1$ starts:

$$4 \rightarrow 5 : P1_1.$$

At $t = 5$, $P2_2$ arrives and has higher priority than $P1$, so $P1$ is preempted:

$$5 \rightarrow 7 : P2_2.$$

$P1_1$ has 2 ms remaining:

$$7 \rightarrow 9 : P1_1.$$

The processor is idle:

$$9 \rightarrow 10 : \text{Idle.}$$

At $t = 10$, $P2_3$ and $P3_2$ arrive. Since $P2$ has higher priority:

$$10 \rightarrow 12 : P2_3.$$

Then:

$$12 \rightarrow 14 : P3_2.$$

The processor is idle:

$$14 \rightarrow 15 : \text{Idle.}$$

At $t = 15$:

$$15 \rightarrow 17 : P_{2_4}.$$

Then the processor is idle until the next hyperperiod:

$$17 \rightarrow 20 : \text{Idle}.$$

10 Complete RMS Gantt Chart

P_2	P_3	P_1	P_2	P_1	Idle	P_2	P_3	Idle	P_2	Idle	
0	2	4	5	7	9	10	12	14	15	17	20

Thus:

$P_2 P_3 P_1 P_2 P_1 Idle P_2 P_3 Idle P_2 Idle$
--

11 Step 7: Deadline Verification

Job	Deadline	Completion	Result
P_{2_1}	5	2	✓
P_{3_1}	10	4	✓
P_{1_1}	20	9	✓
P_{2_2}	10	7	✓
P_{2_3}	15	12	✓
P_{3_2}	20	14	✓
P_{2_4}	20	17	✓

All jobs meet their deadlines.

12 Key Points

- RMS is a fixed-priority scheduling algorithm.
- Shorter period means higher priority.
- A higher-priority process can preempt a lower-priority process.
- The utilization-bound test is sufficient, not necessary.
- For three tasks, the RMS sufficient bound is approximately 77.98%.
- The given task set has utilization 75%, so it passes the RMS sufficient test.
- The hyperperiod is 20 ms.

13 Conclusion

For:

$$P1 = (C = 3, T = 20), \quad P2 = (C = 2, T = 5), \quad P3 = (C = 2, T = 10),$$

$$U = 75\% < 77.98\% = U_{RMS}.$$

Therefore, the task set passes the RMS utilization-bound test and is guaranteed schedulable under the standard assumptions.

14 Is the RMS Utilization-Bound Test Necessary and Sufficient?

No. The standard RMS utilization-bound test is a **sufficient but not necessary** condition for schedulability.

For n periodic tasks, the RMS sufficient utilization bound is:

$$U \leq n(2^{1/n} - 1)$$

where processor utilization is:

$$U = \sum_{i=1}^n \frac{C_i}{T_i}$$

If the condition is satisfied, the task set is guaranteed to be schedulable under RMS, subject to the standard assumptions.

However, if the condition is not satisfied, we **cannot conclude that the task set is unschedulable**. A more precise analysis, such as response-time analysis, may still show that the task set is schedulable.

Therefore:

$$U \leq U_{RMS} \Rightarrow \text{Definitely schedulable}$$

but:

$$U > U_{RMS} \not\Rightarrow \text{Unschedulable}$$

15 Counterexample

Consider the following periodic tasks:

Task	Period T	Execution Time C
$T1$	4	1
$T2$	5	2
$T3$	20	4

Assume:

- all tasks are periodic,

- scheduling is preemptive,
- all tasks are released initially at $t = 0$,
- deadlines are equal to periods,
- context-switch overhead is negligible.

16 Step 1: Calculate Processor Utilization

$$U = \frac{C_1}{T_1} + \frac{C_2}{T_2} + \frac{C_3}{T_3}$$

$$U = \frac{1}{4} + \frac{2}{5} + \frac{4}{20}$$

$$U = 0.25 + 0.40 + 0.20$$

$$\boxed{U = 0.85 = 85\%}$$

17 Step 2: Calculate RMS Utilization Bound

There are:

$$n = 3$$

Therefore:

$$U_{RMS} = 3(2^{1/3} - 1)$$

Since:

$$2^{1/3} \approx 1.2599,$$

$$U_{RMS} = 3(1.2599 - 1)$$

$$U_{RMS} \approx 0.7798$$

Hence:

$$\boxed{U_{RMS} \approx 77.98\%}$$

18 Step 3: Compare the Values

We have:

$$U = 85\%$$

and:

$$U_{RMS} = 77.98\%$$

Therefore:

$$85\% > 77.98\%$$

The task set **fails the RMS utilization-bound test**.

However, this does **not** mean that the task set is unschedulable.

Failure of the utilization-bound test does not prove unschedulability.

19 Step 4: RMS Priority Order

RMS assigns higher priority to the task having the shorter period.

The periods are:

$$T_1 = 4, \quad T_2 = 5, \quad T_3 = 20$$

Therefore:

$$4 < 5 < 20$$

and the priority order is:

$$T1 > T2 > T3$$

Thus, $T1$ has the highest priority and $T3$ has the lowest priority.

20 Step 5: Verify Using Response-Time Analysis

Response-time analysis is more precise than the simple utilization-bound test.

Task $T1$

Since $T1$ has the highest priority:

$$R_1 = C_1$$

$$R_1 = 1$$

Since:

$$1 \leq 4$$

$T1$ meets its deadline.

$$R_1 = 1 \leq 4$$

Task T_2

The response-time equation is:

$$R_2 = C_2 + \left\lceil \frac{R_2}{T_1} \right\rceil C_1$$

Start with:

$$R_2^{(0)} = C_2 = 2$$

First iteration:

$$\begin{aligned} R_2^{(1)} &= 2 + \left\lceil \frac{2}{4} \right\rceil (1) \\ &= 2 + 1 = 3 \end{aligned}$$

Second iteration:

$$\begin{aligned} R_2^{(2)} &= 2 + \left\lceil \frac{3}{4} \right\rceil (1) \\ &= 2 + 1 = 3 \end{aligned}$$

The value has converged:

$$\boxed{R_2 = 3}$$

Since:

$$3 \leq 5$$

T_2 meets its deadline.

Task T_3

The response-time equation is:

$$R_3 = C_3 + \left\lceil \frac{R_3}{T_1} \right\rceil C_1 + \left\lceil \frac{R_3}{T_2} \right\rceil C_2$$

Start with:

$$R_3^{(0)} = C_3 = 4$$

First iteration:

$$\begin{aligned} R_3^{(1)} &= 4 + \left\lceil \frac{4}{4} \right\rceil (1) + \left\lceil \frac{4}{5} \right\rceil (2) \\ &= 4 + 1 + 2 = 7 \end{aligned}$$

Second iteration:

$$R_3^{(2)} = 4 + \left\lceil \frac{7}{4} \right\rceil (1) + \left\lceil \frac{7}{5} \right\rceil (2)$$

$$= 4 + 2 + 4 = 10$$

Third iteration:

$$\begin{aligned} R_3^{(3)} &= 4 + \left\lceil \frac{10}{4} \right\rceil (1) + \left\lceil \frac{10}{5} \right\rceil (2) \\ &= 4 + 3 + 4 = 11 \end{aligned}$$

Fourth iteration:

$$\begin{aligned} R_3^{(4)} &= 4 + \left\lceil \frac{11}{4} \right\rceil (1) + \left\lceil \frac{11}{5} \right\rceil (2) \\ &= 4 + 3 + 6 = 13 \end{aligned}$$

Next iteration:

$$\begin{aligned} R_3^{(5)} &= 4 + \left\lceil \frac{13}{4} \right\rceil (1) + \left\lceil \frac{13}{5} \right\rceil (2) \\ &= 4 + 4 + 6 = 14 \end{aligned}$$

Next iteration:

$$\begin{aligned} R_3^{(6)} &= 4 + \left\lceil \frac{14}{4} \right\rceil (1) + \left\lceil \frac{14}{5} \right\rceil (2) \\ &= 4 + 4 + 6 = 14 \end{aligned}$$

The response time converges to:

$$\boxed{R_3 = 14}$$

Since:

$$14 \leq 20$$

T_3 also meets its deadline.

21 Step 6: Final Response-Time Analysis Result

Task	Response Time R_i	Deadline	Result
T_1	1	4	✓
T_2	3	5	✓
T_3	14	20	✓

All three tasks meet their deadlines.

Therefore:

$$\boxed{\text{The task set is schedulable under RMS.}}$$

Even though:

$$\boxed{U = 85\% > 77.98\%}$$

the task set is schedulable.

22 Why This Is a Counterexample

The example demonstrates:

$$U > U_{RMS} \quad \text{but the task set is still schedulable}$$

Thus, the utilization-bound test is not a necessary condition.

The logical implication is only:

$$U \leq U_{RMS} \Rightarrow \text{schedulable}$$

The reverse implication does not hold:

$$\text{schedulable} \not\Rightarrow U \leq U_{RMS}$$

23 Summary

Condition	Interpretation
$U \leq U_{RMS}$	Guaranteed schedulable under RMS
$U > U_{RMS}$	Utilization test is inconclusive
$U > U_{RMS}$ and response-time analysis passes	Schedulable
$U > U_{RMS}$ and response-time analysis fails	Not schedulable under RMS

For the counterexample:

$$U = 85\%, \quad U_{RMS} = 77.98\%$$

The utilization test fails, but response-time analysis gives:

$$R_1 = 1, \quad R_2 = 3, \quad R_3 = 14$$

and:

$$1 \leq 4, \quad 3 \leq 5, \quad 14 \leq 20.$$

Therefore:

$$\text{All deadlines are met.}$$

$$\text{RMS utilization-bound test is sufficient but not necessary.}$$

Earliest Deadline First (EDF) Scheduling

Explanation and Step-by-Step Example

Prashant Jadav
DEPSTAR, CHARUSAT

1 Period and Deadline

In real-time scheduling, the **period** and **deadline** are separate parameters. They may be equal or different.

A periodic task can be represented as:

$$T_i = (P_i, C_i, D_i)$$

where:

- P_i = Period: time between successive releases of the task.
- C_i = Execution time: CPU time required by one job.
- D_i = Relative deadline: maximum allowed time from release to completion.

1.1 Common Case: Deadline Equals Period

Many textbook scheduling problems assume:

$$D_i = P_i$$

For example:

$$T_1 = (P = 2, C = 0.9, D = 2)$$

The successive jobs are:

Job	Release Time	Absolute Deadline
$T_{1,1}$	0	2
$T_{1,2}$	2	4
$T_{1,3}$	4	6
$T_{1,4}$	6	8

1.2 Period and Deadline Can Be Different

For example:

$$T_1 = (P = 5, C = 1, D = 3)$$

The task is released every 5 ms, but each job must complete within 3 ms of its release.

Thus:

Job	Release Time	Absolute Deadline
$T_{1,1}$	0	3
$T_{1,2}$	5	8
$T_{1,3}$	10	13

The absolute deadline of a job is:

$$d_{i,k} = r_{i,k} + D_i$$

where $r_{i,k}$ is its release time.

2 Earliest Deadline First (EDF)

EDF is a **dynamic-priority scheduling algorithm**.

At every scheduling decision, EDF selects the ready job having the earliest **absolute deadline**.

Earlier absolute deadline $\Rightarrow$ Higher priority

This is different from Rate Monotonic Scheduling (RMS), where priority is based on the period.

	RMS	EDF
Priority basis	Period	Absolute deadline
Priority type	Static	Dynamic
Higher priority	Shorter period	Earlier deadline

3 EDF Utilization Test

For the standard periodic-task model with deadlines equal to periods, EDF can guarantee schedulability when:

$$U \leq 1$$

where:

$$U = \sum_{i=1}^n \frac{C_i}{P_i}$$

Therefore, EDF can theoretically utilize up to 100% of the processor under the standard assumptions.

4 Step-by-Step EDF Example

Consider:

$$T_1 = (P = 2, C = 0.9, D = 2)$$

$$T_2 = (P = 5, C = 2.3, D = 5)$$

Assume:

- Both tasks are released at $t = 0$.

- Scheduling is preemptive.
- Relative deadlines equal periods.

4.1 Step 1: Calculate Hyperperiod

$$H = \text{LCM}(2, 5) = 10 \text{ ms}$$

Therefore, we construct the schedule from 0 to 10 ms.

4.2 Step 2: Determine Jobs and Deadlines

Job	Release	C	Deadline
$T_{1,1}$	0	0.9	2
$T_{1,2}$	2	0.9	4
$T_{1,3}$	4	0.9	6
$T_{1,4}$	6	0.9	8
$T_{1,5}$	8	0.9	10
$T_{2,1}$	0	2.3	5
$T_{2,2}$	5	2.3	10

5 EDF Execution Trace

At $t = 0$

Ready jobs:

$$T_{1,1} : D = 2, \quad T_{2,1} : D = 5$$

EDF chooses $T_{1,1}$ because $2 < 5$.

$$0 \rightarrow 0.9 : T_{1,1}$$

At $t = 0.9$

Only $T_{2,1}$ is ready.

$$0.9 \rightarrow 2.0 : T_{2,1}$$

It executes for:

$$2 - 0.9 = 1.1 \text{ ms}$$

Remaining execution time:

$$2.3 - 1.1 = 1.2 \text{ ms}$$

At $t = 2$

$T_{1,2}$ arrives with deadline:

$$D = 4$$

The remaining $T_{2,1}$ has deadline 5.

Since:

$$4 < 5$$

EDF preempts $T_{2,1}$.

$$2 \rightarrow 2.9 : T_{1,2}$$

At $t = 2.9$

$T_{2,1}$ resumes.

$$2.9 \rightarrow 4.1 : T_{2,1}$$

It completes.

At $t = 4.1$

$T_{1,3}$ has deadline 6.

$$4.1 \rightarrow 5.0 : T_{1,3}$$

It completes at $t = 5$.

At $t = 5$

$T_{2,2}$ arrives with deadline 10.

$$5 \rightarrow 6.0 : T_{2,2}$$

It executes for 1 ms.

At $t = 6$

$T_{1,4}$ arrives with deadline 8.

Since:

$$8 < 10$$

EDF preempts $T_{2,2}$.

$$6 \rightarrow 6.9 : T_{1,4}$$

At $t = 6.9$

$T_{2,2}$ resumes.

$$6.9 \rightarrow 8.0 : T_{2,2}$$

It has executed:

$$1 + 1.1 = 2.1 \text{ ms}$$

Remaining:

$$2.3 - 2.1 = 0.2 \text{ ms}$$

At $t = 8$

$T_{1,5}$ arrives with deadline 10.

Both $T_{1,5}$ and the remaining part of $T_{2,2}$ have deadline 10. Using the tie rule that allows the currently executing job to continue:

$$8.0 \rightarrow 8.2 : T_{2,2}$$

$T_{2,2}$ completes.

At $t = 8.2$

Execute $T_{1,5}$:

$$8.2 \rightarrow 9.1 : T_{1,5}$$

The processor is idle from 9.1 to 10.

$$9.1 \rightarrow 10 : \text{Idle}$$

6 Complete EDF Gantt Chart

T_1	T_2	T_1	T_2	T_1	T_2	T_1	T_2	T_1	Idle	
0	0.9	2	2.9	4.1	5	6	6.9	8.2	9.1	10

Thus:

T_1	T_2	T_1	T_2	T_1	T_2	T_1	T_2	T_1	Idle
-------	-------	-------	-------	-------	-------	-------	-------	-------	------

with the exact time intervals:

Interval	Execution
0 – 0.9	$T_{1,1}$
0.9 – 2.0	$T_{2,1}$
2.0 – 2.9	$T_{1,2}$
2.9 – 4.1	$T_{2,1}$
4.1 – 5.0	$T_{1,3}$
5.0 – 6.0	$T_{2,2}$
6.0 – 6.9	$T_{1,4}$
6.9 – 8.2	$T_{2,2}$
8.2 – 9.1	$T_{1,5}$
9.1 – 10	Idle

7 Deadline Check

Job	Deadline	Completion	Result
$T_{1,1}$	2	0.9	✓
$T_{1,2}$	4	2.9	✓
$T_{2,1}$	5	4.1	✓
$T_{1,3}$	6	5.0	✓
$T_{1,4}$	8	6.9	✓
$T_{2,2}$	10	8.2	✓
$T_{1,5}$	10	9.1	✓

All deadlines are met.

8 Utilization

The processor utilization is:

$$U = \frac{0.9}{2} + \frac{2.3}{5}$$

$$U = 0.45 + 0.46 = 0.91$$

Therefore:

$$U = 91\%$$

For the standard EDF model:

$$U \leq 1$$

Since:

$$0.91 < 1$$

the task set passes the EDF utilization test.

The task set is schedulable under EDF.

9 Key Points

1. Period and deadline are separate parameters and may be equal or different.
2. RMS uses the period to determine priority.
3. EDF uses the absolute deadline to determine priority.
4. EDF has dynamic priorities.
5. For the standard periodic-task model with $D_i = P_i$, EDF can guarantee schedulability when total utilization is at most 100%.
6. In the example, utilization is 91%, so the task set passes the EDF utilization test.
7. The complete schedule is constructed over the hyperperiod:

$$H = \text{LCM}(2, 5) = 10 \text{ ms.}$$